# Initialisation d'une Application React et Découverte de l'Environnement

## Objectifs du TP

Au cours de cet atelier pratique, vous allez :

1. Vérifier votre environnement d'exécution Node.js et npm en ligne de commande.
2. Initialiser une Single Page Application React moderne avec l'outil de build Vite.
3. Analyser le fichier manifeste `package.json` et le cycle d'amorçage (`index.html`, `main.jsx`, `App.jsx`).
4. Nettoyer le code de démonstration pour construire une première vue déclarative interactive.
5. Inspecter le DOM Virtuel et comparer une mise à jour impérative avec une mise à jour déclarative.
6. Compiler l'application pour la production et analyser le bundle généré.

---

## Contexte du Projet Fil Rouge

Pour cette première mise en pratique, vous êtes chargés d'initialiser le socle technique d'un portail de supervision de stations d'observation baptisé **OmniMonitor**. Votre mission consiste à mettre en place l'architecture de base du projet, à comprendre le rôle de chaque fichier généré et à afficher un premier tableau de bord fonctionnel.

---

## Étape 1 : Vérification de l'Environnement Système

Avant de créer le projet, assurez-vous que l'usine logicielle locale (Node.js et npm) est opérationnelle sur votre poste de travail.

1. Ouvrez un terminal (PowerShell, Git Bash ou Terminal intégré à VS Code).
2. Exécutez les commandes suivantes pour vérifier les versions installées :

```shell
node --version
npm --version
```

3. Vérifiez que la version de Node.js affichée est supérieure ou égale à la version 18 (idéalement v20 ou v22 LTS).
4. Créez un dossier de travail dédié aux travaux pratiques de la formation et déplacez-vous à l'intérieur :

```shell
mkdir formation-react
cd formation-react
```

---

## Étape 2 : Création du Projet avec Vite

Vous allez maintenant générer le squelette de la Single Page Application à l'aide de Vite.

1. Dans votre terminal, lancez la commande d'initialisation suivante :

```shell
npm create vite@latest omnimonitor-app -- --template react
```

2. Si npm vous demande confirmation pour installer le paquet `create-vite`, tapez `y` puis validez avec Entrée.
3. Une fois l'arborescence créée, entrez dans le répertoire du projet et ouvrez-le dans Visual Studio Code :

```shell
cd omnimonitor-app
code .
```

4. Observez la liste des fichiers générés dans l'explorateur à gauche. Notez que le dossier `node_modules` n'existe pas encore : seuls les fichiers sources et le manifeste `package.json` sont présents.
5. Installez maintenant les dépendances listées dans `package.json` :

```shell
npm install
```

6. Observez l'apparition du dossier `node_modules/` et du fichier de verrouillage `package-lock.json`.

---

## Étape 3 : Analyse de la Configuration et Démarrage à Chaud (HMR)

1. Ouvrez le fichier `package.json` et identifiez :
    - Les bibliothèques présentes dans `dependencies` (`react` et `react-dom`).
    - Les outils présents dans `devDependencies` (`vite`, `@vitejs/plugin-react`, `eslint`).
    - Les commandes disponibles dans la section `scripts`.
2. Démarrez le serveur de développement local en exécutant :

```
Shell
npm run dev
```

3. Ouvrez votre navigateur à l'adresse locale affichée dans le terminal (généralement `http://localhost:5173`).
4. Ouvrez les outils de développement du navigateur (touche `F12` ou clic droit puis **Inspecter**) :
    - Dans l'onglet **Elements** (ou Inspecteur), dépliez la balise `<div id="root">`. Observez que tout le HTML de l'application a été injecté dynamiquement par JavaScript à l'intérieur de ce conteneur.
    - Effectuez un clic droit sur la page et choisissez **Afficher le code source de la page** (`Ctrl+U`). Comparez ce code source brut (qui correspond au fichier `index.html` initial presque vide) avec l'arbre DOM actif visible dans l'inspecteur d'éléments.

---

## Étape 4 : Nettoyage du Socle et Cycle d'Amorçage

Le modèle par défaut généré par Vite contient un compteur de démonstration et des styles inutiles pour notre application métier. Vous allez épurer le projet.

1. Supprimez le fichier `src/App.css`.

2. Remplacez l'intégralité du contenu du fichier `src/index.css` par la feuille de style de base suivante :

```css
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: system-ui, -apple-system, sans-serif;
  background-color: #f4f6f8;
  color: #1e293b;
  line-height: 1.5;
  padding: 2rem;
}

.container {
  max-width: 900px;
  margin: 0 auto;
  background: #ffffff;
  padding: 2rem;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
}

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 2px solid #e2e8f0;
  padding-bottom: 1rem;
  margin-bottom: 1.5rem;
}

.badge {
  padding: 0.35rem 0.75rem;
  border-radius: 4px;
  font-weight: 600;
  font-size: 0.875rem;
}

.badge-online {
  background-color: #dcfce7;
  color: #166534;
}
```

```
.badge-offline {
  background-color: #fee2e2;
  color: #991b1b;
}

.btn {
  padding: 0.6rem 1.2rem;
  border: none;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
  background-color: #2563eb;
  color: white;
  margin-top: 1rem;
}

.btn:hover {
  background-color: #1d4ed8;
}

.info-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
  margin-top: 1.5rem;
}

.card {
  background: #f8fafc;
  border: 1px solid #e2e8f0;
  padding: 1rem;
  border-radius: 6px;
}
```

3. Ouvrez le fichier `index.html` situé à la racine du projet :
    - Modifiez l'attribut `lang="en"` en `lang="fr"`.
    - Remplacez le contenu de la balise `<title>` par `OmniMonitor - Supervision Temps Réel`.
4. Ouvrez le fichier `src/main.jsx` et vérifiez que le montage sur l'élément `#root` est bien encapsulé dans `<StrictMode>`.

---

## Étape 5 : Mise en Pratique du Paradigme Déclaratif

Vous allez maintenant coder le composant racine `src/App.jsx` en appliquant le principe fondamental vu en cours : l'interface est une projection directe de l'état (`UI = f(state)`), sans aucune sélection manuelle de nœud DOM (`document.getElementById` ou `querySelector` sont proscrits).

1. Ouvrez `src/App.jsx` et remplacez tout son contenu pour concevoir une vue qui affiche :

    - Le nom de la plateforme : `OmniMonitor v1.0`.
    - Un état booléen `isSystemOnline` initialisé à `true` (en utilisant le Hook `useState` importé depuis `'react'`).
    - Un badge visuel qui affiche `Système Opérationnel` avec la classe CSS `badge badge-online` lorsque `isSystemOnline` vaut `true`, ou `Maintenance en cours` avec la classe `badge badge-offline` lorsqu'il vaut `false`.
    - Trois cartes d'informations statiques dans une grille (`.info-grid`) présentant :
        - Carte 1 : Nombre de capteurs actifs (`42` si en ligne, `0` si hors ligne).
        - Carte 2 : Mode de synchronisation (`Temps réel` si en ligne, `Suspendu` si hors ligne).
        - Carte 3 : Environnement d'exécution (`Développement (Vite ESM)`).
    - Un bouton permettant de basculer l'état `isSystemOnline` entre `true` et `false` au clic. Le texte du bouton doit s'adapter automatiquement (`Passer en mode maintenance` ou `Relancer la supervision`).
2. Enregistrez votre fichier (`Ctrl+S`).

3. Sans rafraîchir manuellement votre navigateur, observez la mise à jour instantanée de la page grâce au **Hot Module Replacement (HMR)** de Vite.

4. Cliquez plusieurs fois sur le bouton tout en observant l'onglet **Elements** des outils de développement Chrome/Firefox :

    - Repérez les nœuds HTML qui clignotent dans l'inspecteur lors du clic.
    - Constatez que React ne détruit ni ne recrée toute la page `<div class="container">`, mais modifie chirurgicalement uniquement les textes et l'attribut `class` qui ont réellement changé (principe de **Réconciliation**).

---

## Étape 6 : Inspection avec React Developer Tools et Test de StrictMode

1. Ajoutez une instruction `console.log('Rendu du composant App - Statut :', isSystemOnline);` juste avant le `return` de votre fonction `App` dans `src/App.jsx`.
2. Ouvrez l'onglet **Console** des outils de développement de votre navigateur et rafraîchissez la page (`F5`).
3. **Question d'analyse :** Pourquoi le message s'affiche-t-il **deux fois** consécutives dans la console lors du chargement initial ou à chaque clic ?
    - Vérifiez votre hypothèse en commentant temporairement les balises `<StrictMode>` dans `src/main.jsx`, puis rétablissez-les.
4. Ouvrez l'onglet **Components** de l'extension **React Developer Tools** dans votre navigateur :
    - Sélectionnez le composant `App`.
    - Dans le panneau de droite, repérez la section `hooks` -> `State`.
    - Cochez et décochez directement la valeur booléenne depuis l'extension React DevTools et observez l'interface se synchroniser immédiatement.

---

## Étape 7 : Compilation de Production (Build) et Prévisualisation

Pour terminer ce premier atelier, vous allez vérifier comment Vite et Rollup transforment votre code source JSX modulaire en fichiers statiques optimisés prêts à être déployés sur un serveur web de production.

1. Dans votre terminal, arrêtez le serveur de développement en appuyant sur `Ctrl+C`.
2. Lancez la commande de compilation de production :

```shell
npm run build
```

3. Observez le rapport de compilation affiché dans le terminal (taille des fichiers générés et taille après compression gzip).
4. Dépliez le nouveau dossier `dist/` qui vient d'apparaître à la racine de votre projet :
    - Ouvrez `dist/index.html` : remarquez que l'appel vers `/src/main.jsx` a été remplacé par des balises pointant vers les fichiers compilés et hachés dans `/assets/index-[hash].js` et `/assets/index-[hash].css`.

    - Ouvrez le fichier JavaScript généré dans `dist/assets/` : le code JSX a été entièrement transpilé en JavaScript standard minifié, et le code de `react` et `react-dom` y a été assemblé.
5. Testez le rendu exact de ce build de production localement en lançant :

```shell
npm run preview
```

6. Ouvrez l'URL indiquée (généralement `http://localhost:4173`), ouvrez la console du navigateur et constatez que le double affichage lié à `<StrictMode>` a disparu en mode production.
7. Arrêtez le serveur de prévisualisation (`Ctrl+C`) et relancez `npm run dev` pour être prêt pour la session suivante.

---

## Critères de Validation

Avant de passer à la suite, vérifiez que vous avez validé les points suivants :

- Le projet démarre sans erreur avec `npm run dev`.
- Le fichier `src/App.jsx` n'utilise aucune manipulation impérative du DOM (`document.getElementById` absent).
- Le changement d'état met à jour simultanément le badge d'état, les compteurs des cartes et le libellé du bouton.
- Vous savez expliquer la différence entre le dossier `src/` (code de développement) et le dossier `dist/` (bundle compilé pour la production).